

Adaptive Boosting & Gradient Boosting

A Step-by-Step Tutorial with Worked Numerical Examples and Python

From weak learners to state-of-the-art ensembles — the intuition, the mathematics, a pencil-and-paper example you can verify by hand, and complete, runnable code.

A Data Science Tutorial

Companion Jupyter notebook: **`Boosting_Tutorial.ipynb`**

1. The Big Idea: Weak Learners into a Strong Learner

Boosting is one of the most powerful ideas in machine learning. A large share of winning Kaggle solutions and production systems for tabular data are boosted tree ensembles — XGBoost, LightGBM, CatBoost. Yet the core idea is simple enough to work out with a pencil, which is exactly what this tutorial does.

A **weak learner** is a model only slightly better than random guessing. The classic example is a **decision stump**: a decision tree with a single split. Boosting builds a strong model as a **weighted sum of many weak learners trained sequentially**:

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

Every boosting algorithm answers one question: *“Where is the current ensemble still wrong, and how do I build the next model to fix exactly that?”*

- **AdaBoost** answers by putting more **weight** on the samples that are currently misclassified.
- **Gradient Boosting** answers by fitting the next model to the **residuals** — the negative gradient of the loss, i.e. literally “what is left over.”

Two knobs shared by all boosting methods matter enormously in practice:

Knob	Symbol	Effect
Number of rounds	M	More rounds lower bias, but eventually overfit
Learning rate (shrinkage)	nu	Smaller = slower, more robust learning (needs larger M)

2. AdaBoost (Adaptive Boosting)

2.1 Intuition

Imagine a study group preparing for an exam. After each practice test, the group spends **more time on the questions they got wrong**. AdaBoost does exactly this with training samples:

- Start with every sample equally important (equal weights).
- Train a weak learner; see which samples it gets wrong.
- **Increase the weight** of misclassified samples and **decrease** the weight of correct ones.
- Train the next weak learner on the re-weighted data — it must focus on the hard cases.
- Give each weak learner a say (α) proportional to its accuracy.
- Final prediction = weighted majority vote.

2.2 The Algorithm, Step by Step

We use labels $y \in \{-1, +1\}$, which makes the math clean. Given data and a number of rounds M :

Step 0 — Initialize weights equally (for all i):

$$w_i^{(1)} = \frac{1}{n}$$

Step 1 — Train a weak learner $h_m(x)$ on the data weighted by $w^{(m)}$.

Step 2 — Weighted error of that learner:

$$\epsilon_m = \frac{\sum_i w_i^{(m)} \mathbb{I}[h_m(x_i) \neq y_i]}{\sum_i w_i^{(m)}}$$

Step 3 — Learner weight (its “say”). Accurate learners get a large α ; a coin-flip learner ($\epsilon = 0.5$) gets $\alpha = 0$:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right)$$

Step 4 — Update sample weights, then renormalize so they sum to 1. Correct samples ($y \cdot h = +1$) shrink by $e^{-\alpha}$; wrong samples ($y \cdot h = -1$) grow by $e^{+\alpha}$:

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))}{Z_m}$$

Output — the strong classifier is the weighted vote:

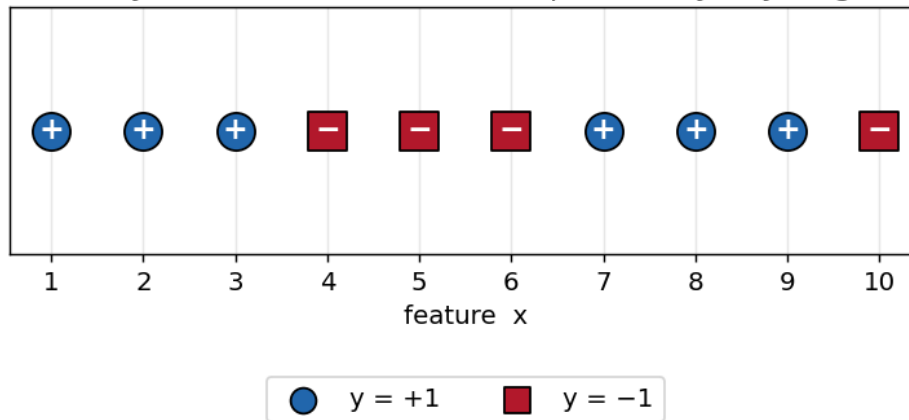
$$F(x) = \text{sign}\left(\sum_{m=1}^M \alpha_m h_m(x)\right)$$

2.3 A Fully Worked Numerical Example

Ten points on a line, with labels that **no single stump can separate** — so we genuinely need boosting. The weak learners are stumps of the form “predict +1 on one side of a threshold, -1 on the other.”

x	1	2	3	4	5	6	7	8	9	10
y	+1	+1	+1	-1	-1	-1	+1	+1	+1	-1

AdaBoost toy dataset: labels are not separable by any single stump



The dataset: positives (blue circles) surround a negative block (4–6), plus a lone negative at x = 10.

Round 1

All weights equal 0.1. The best stump is “predict +1 if $x < 3.5$, else -1.” It misclassifies points 7, 8, 9 (they are +1 but land on the -1 side).

$$\varepsilon_1 = 3 \times 0.1 = 0.30, \quad \alpha_1 = \frac{1}{2} \ln \frac{0.7}{0.3} = 0.4236$$

Weight update. Correct samples: $0.1 \cdot e^{-0.4236} = 0.0655$. Wrong samples (7, 8, 9): $0.1 \cdot e^{+0.4236} = 0.1528$. Normalizer $Z_1 = 7(0.0655) + 3(0.1528) = 0.9165$. After dividing: correct $\rightarrow$ **0.0714**, wrong $\rightarrow$ **0.1667**. The three hard points are now the heaviest.

Round 2

With the new weights, the best stump is “+1 if $x < 9.5$, else -1” (it now gets 7, 8, 9 right). It misclassifies 4, 5, 6.

$$\varepsilon_2 = 3 \times 0.0714 = 0.2143, \quad \alpha_2 = 0.6496$$

Round 3

The best stump is now “+1 if $x \geq 6.5$, else -1 .” It misclassifies 1, 2, 3, 10.

$$\varepsilon_3 = 0.1818, \quad \alpha_3 = 0.7520$$

Combine — the weighted vote

$$F(x) = \text{sign}(0.4236 h_1 + 0.6496 h_2 + 0.7520 h_3)$$

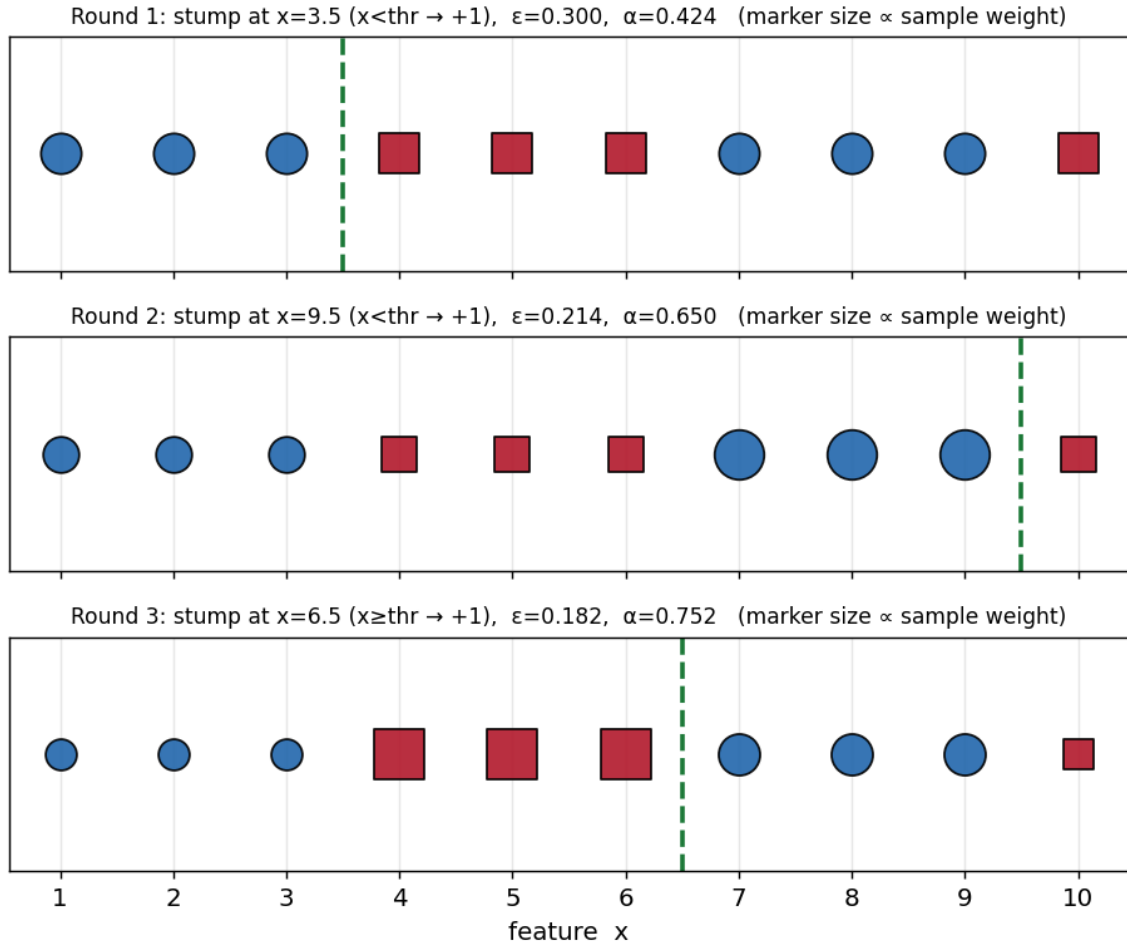
Check $x = 7$: $h_1 = -1$, $h_2 = +1$, $h_3 = +1 \rightarrow F = -0.4236 + 0.6496 + 0.7520 = +0.978 \rightarrow +1 \checkmark$

Check $x = 1$: $h_1 = +1$, $h_2 = +1$, $h_3 = -1 \rightarrow F = 0.4236 + 0.6496 - 0.7520 = +0.321 \rightarrow +1 \checkmark$

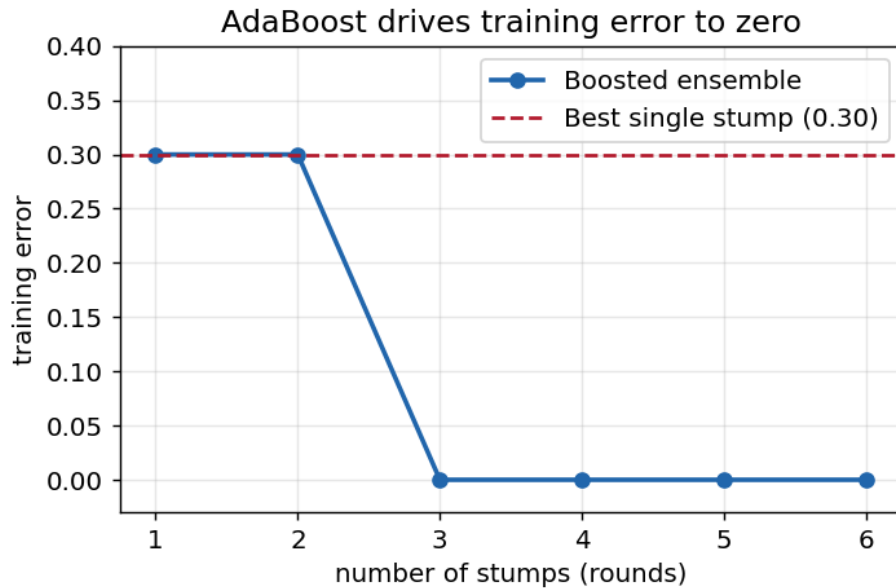
The punchline

Three stumps — each individually wrong on 30% of the data — combine into a classifier that is **100% correct**. That is boosting in a nutshell.

How AdaBoost re-weights samples each round



How the sample weights evolve. Marker size is proportional to weight; the misclassified points visibly grow so the next stump is forced to attend to them.



Training error of the growing ensemble versus the best single stump.

2.4 AdaBoost from Scratch (Python)

This code reproduces every number above exactly.

```
import numpy as np

X = np.array([1,2,3,4,5,6,7,8,9,10], dtype=float)
y = np.array([1,1,1,-1,-1,-1,1,1,1,-1], dtype=float)
n = len(X)

def best_stump(X, y, w):
    """Decision stump (threshold + polarity) with lowest weighted error."""
    best = None
    thresholds = [X.min()-0.5] + [(X[i]+X[i+1])/2 for i in range(n-1)] + [X.max()+0.5]
    for thr in thresholds:
        for pol in (+1, -1):
            # +1: '+1 if x<thr'; -1: '+1 if x>=thr'
            pred = np.where((X < thr) if pol == 1 else (X >= thr), 1.0, -1.0)
            err = np.sum(w[pred != y])
            if best is None or err < best['err']:
                best = dict(thr=thr, pol=pol, err=err, pred=pred)
    return best

w, F = np.ones(n)/n, np.zeros(n)
for t in range(1, 4):
    s = best_stump(X, y, w)
    eps = s['err']
    alpha = 0.5 * np.log((1 - eps) / eps)
    w = w * np.exp(-alpha * y * s['pred']) # re-weight ...
    w = w / w.sum() # ... and normalize
    F += alpha * s['pred']
    print(f"Round {t}: eps={eps:.4f} alpha={alpha:.4f}")

print("Predictions:", np.sign(F).astype(int)) # -> matches y exactly (100% accuracy)
```

2.5 The scikit-learn Version

In practice you would simply use scikit-learn. Its SAMME algorithm is the multi-class generalization of the discrete AdaBoost derived above.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1), # a decision stump
    n_estimators=10, learning_rate=1.0, random_state=0,
)
clf.fit(X.reshape(-1, 1), (y > 0).astype(int))
print("accuracy:", clf.score(X.reshape(-1, 1), (y > 0).astype(int))) # 1.0
```

3. Gradient Boosting

3.1 Intuition

AdaBoost re-weights samples. Gradient Boosting takes a more general view: **fit each new model to the errors that remain — the residuals**. Start with a simple prediction (the mean). Look at how far off you are on each point (the residual $y - F(x)$). Train a small tree to **predict those residuals**, add a shrunk version of it to the model, and repeat. Each tree nudges the prediction a little closer to the truth.

Why “gradient”?

For squared-error loss, the residual $y - F(x)$ is exactly the **negative gradient** of the loss with respect to the prediction. So “fit the residuals” is really “take a gradient-descent step in the space of functions.” That is why the method extends to *any* differentiable loss — logistic, Huber, and more.

3.2 The Algorithm, Step by Step (regression, squared-error loss)

With loss $L(y, F) = \frac{1}{2}(y - F)^2$, the negative gradient equals the residual ($y - F$).

Step 0 — Initialize with the best constant (the mean for squared error):

$$F_0(x) = \bar{y}$$

Step 1 — Compute residuals (pseudo-residuals):

$$r_i^{(m)} = y_i - F_{m-1}(x_i)$$

Step 2 — Fit a regression tree $h_m(x)$ to the residuals.

Step 3 — Update with learning rate $\nu \in (0, 1]$:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

The learning rate shrinks each tree's contribution. Small ν (e.g. 0.1) plus many trees almost always generalizes better than a few full-strength trees.

3.3 A Fully Worked Numerical Example

Ten points, one feature x , continuous target y . We use learning rate $\nu = 0.5$ and regression stumps (one split, predicting the mean of each side).

x	1	2	3	4	5	6	7	8	9	10
y	1.2	1.9	3.1	3.9	6.2	6.0	7.1	8.9	9.2	10.5

Step 0. $F_0 = \text{mean}(y) = 58.0 / 10 = 5.8$. Initial SSE = $\sum(y_i - 5.8)^2 = 92.42$.

Round 1. Residuals $r = y - 5.8 = [-4.6, -3.9, -2.7, -1.9, 0.4, 0.2, 1.3, 3.1, 3.4, 4.7]$. Best split $x < 4.5$: left mean = $-13.1/4 = -3.275$, right mean = $13.1/6 = +2.183$. Update $F_1 = 5.8 + 0.5 \cdot h_1$. For $x = 1$: $5.8 + 0.5(-3.275) = 4.1625$. New SSE = 38.79.

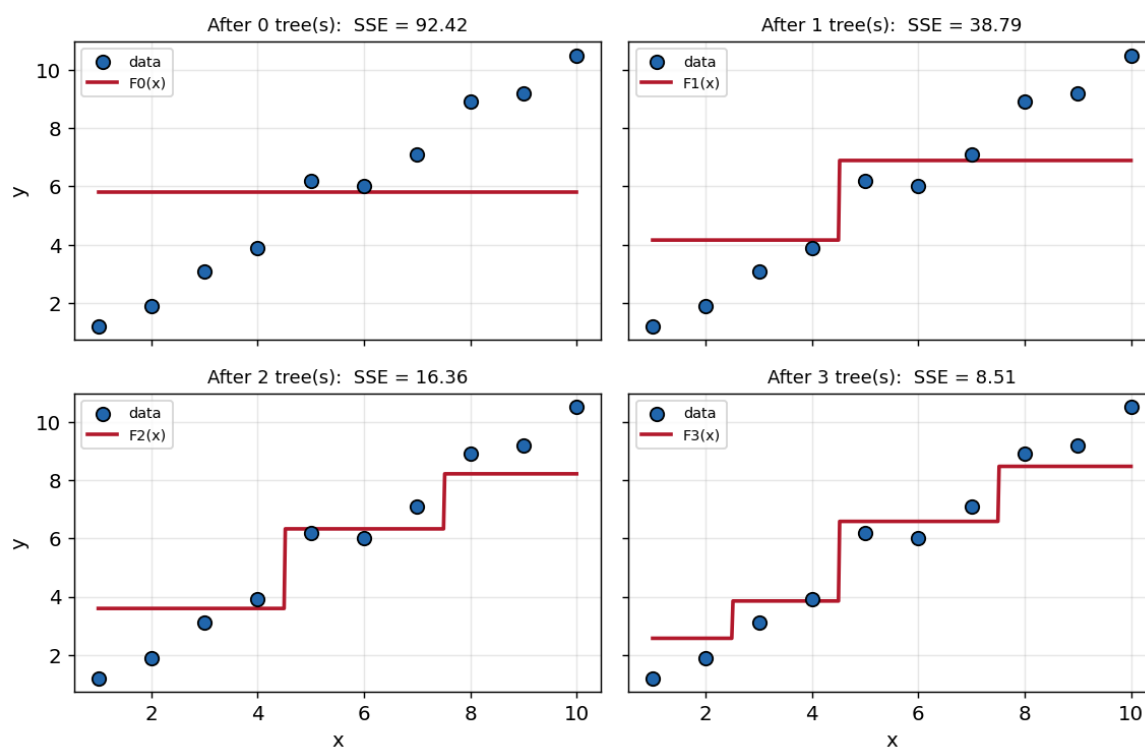
Round 2. New residuals $r = y - F_1$. Best split $x < 7.5$ (left -1.132 , right $+2.642$). SSE = 16.36.

Round 3. Best split $x < 2.5$ (left -2.046 , right $+0.512$). SSE = 8.51.

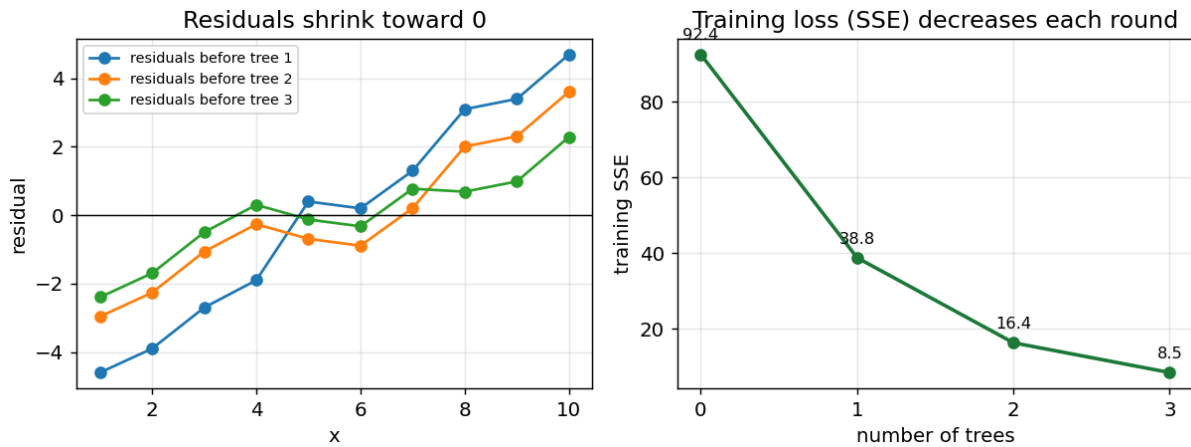
Training error more than halves each round

92.42 $\rightarrow$ 38.79 $\rightarrow$ 16.36 $\rightarrow$ 8.51. Each small tree removes a chunk of the error that the previous ensemble left behind.

Gradient Boosting builds the fit stage by stage (learning rate = 0.5)



The fit built stage by stage. F_0 is a flat line at the mean; each added stump introduces one more step, tracking the data more closely.



Left: residuals shrink toward zero. Right: training SSE falls at every round.

3.4 Gradient Boosting from Scratch (Python)

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

class GradientBoostRegressor:
    """Gradient boosting for squared-error loss with shallow regression trees."""
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=2):
        self.M, self.nu, self.max_depth = n_estimators, learning_rate, max_depth
        self.trees = []

    def fit(self, X, y):
        self.F0 = y.mean()
        F = np.full(len(y), self.F0)
        for _ in range(self.M):
            residual = y - F  # negative gradient (squared error)
            tree = DecisionTreeRegressor(max_depth=self.max_depth).fit(X, residual)
            F = F + self.nu * tree.predict(X)  # shrunken update
            self.trees.append(tree)
        return self

    def predict(self, X):
        F = np.full(X.shape[0], self.F0)
        for tree in self.trees:
            F += self.nu * tree.predict(X)
        return F
```

3.5 The scikit-learn Version

```
from sklearn.ensemble import GradientBoostingRegressor

gbr = GradientBoostingRegressor(n_estimators=50, learning_rate=0.3,
                                max_depth=1, random_state=0)

gbr.fit(X.reshape(-1, 1), y)
print(gbr.predict(X.reshape(-1, 1)))
```

4. Comparison & Practical Guidance

4.1 AdaBoost vs. Gradient Boosting

	AdaBoost	Gradient Boosting
Core mechanism	Re-weights samples	Fits tree to residuals / gradient
Loss function	Exponential (implicit)	Any differentiable loss
How trees combine	Weighted vote (weight α)	Additive, shrunk by v
Typical weak learner	Stump (depth 1)	Shallow tree (depth 2–8)
Sensitivity to outliers	High	Lower (esp. robust losses)
Modern descendants	—	XGBoost, LightGBM, CatBoost

Rule of thumb: reach for gradient boosting (or XGBoost/LightGBM) first — it is more flexible and usually more accurate. AdaBoost remains a beautifully simple mental model and works well on clean data.

4.2 The Learning-Rate / Number-of-Trees Trade-off

The two knobs interact. A large learning rate drops error fast but plateaus higher and can overfit; small learning rates (0.05–0.1) need more trees but reach a **lower** error. The classic recipe: **set a small learning rate, then use early stopping** to choose the number of trees. The companion notebook plots this directly on a real dataset.

4.3 Practical Checklist

- **Start** with GradientBoosting (or XGBoost / LightGBM for larger data).
- **Tune in this order:** learning_rate (small) → n_estimators (via early stopping) → max_depth (3–6) → subsampling / regularization.
- **No feature scaling needed** — tree-based boosting is invariant to monotone feature transforms.
- **Watch for overfitting:** monitor a validation curve; more trees is not always better.
- **Noisy labels?** Prefer gradient boosting with a robust loss (Huber) over AdaBoost.

Summary

- **Boosting** turns many weak learners into one strong learner by training them sequentially, each fixing the previous ensemble's mistakes.

- **AdaBoost** re-weights samples; the weight $\alpha = \frac{1}{2} \cdot \ln((1-\epsilon)/\epsilon)$ gives accurate learners more say. Our three-stump example reached 100% accuracy on data no single stump could split.
 - **Gradient Boosting** fits each new tree to the residuals (the negative gradient), generalizing to any differentiable loss. Our example cut SSE from 92.4 to 8.5 in three tiny trees.
 - The **learning rate** and **number of trees** trade off speed, accuracy, and overfitting.
-

All numbers in this tutorial are reproduced exactly by the companion notebook **Boosting_Tutorial.ipynb**. Open it and run every cell top to bottom to experiment with your own data.